Observer

Observer · Fan out video.published to audit+UI · Build with Waqar

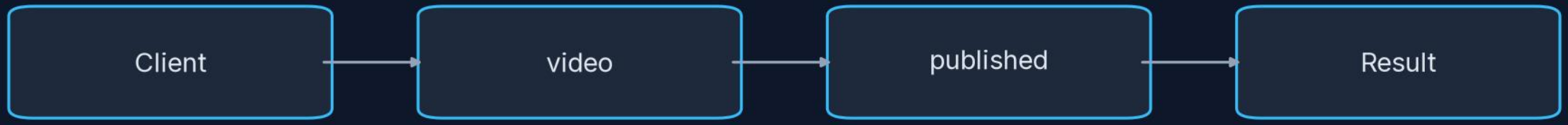
Today's Hook

- Today's force: Fan out video.published to audit+UI
- We earn Observer by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Listener, EventBus

The Problem

- Scenario: Fan out video.published to audit+UI
- Constraints: solve «Fan out video.published to audit+UI» with Observer — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Fan out video.published to audit+UI
- Success looks like: audit v1

Structure



Observer: Fan out video.published to audit+UI

Real-World Scenario

- Running example: Fan out video.published to audit+UI
- Key types: Listener, EventBus
- Happy-path stdout cue: audit v1
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.gof.behavioral.observer.problem;

import java.util.*;

public final class NaivePublish {
  public static void publish(String id){ System.out.println("audit "+id); System.out.println("ui "+id); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Fan out video.published to audit+UI
- Without Observer, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.observer.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

interface Listener { void on(String event, String id); }
final class EventBus {
  private final Map<String, List<Listener>> subs = new HashMap<>();
  void on(String event, Listener l){ subs.computeIfAbsent(event,k->new ArrayList<>()).add(l); }
  void publish(String event, String id){
    for (var l: subs.getOrDefault(event, List.of())) l.on(event, id);
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.observer.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Observer: video.published fan-out ===");
    System.out.println("--- naive ---"); NaivePublish.publish("v1");
System.out.println("--- observer ---");
var bus=new EventBus(); List<String> log=new ArrayList<>();
bus.on("video.published", (e,id)->log.add("audit:"+id));
bus.on("video.published", (e,id)->log.add("ui:"+id));
bus.publish("video.published","v1");
bus.publish("other","x");
System.out.println(log);
  }
}
```

Walkthrough Output

```
=== Observer: video.published fan-out ===
--- naive ---
audit v1
ui v1
--- observer ---
[audit:v1, ui:v1]
```

Step-by-Step Trace

- Match stdout to stages — look for: audit v1
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 21-observer ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Observer
// Prefer records + sealed where they shrink the demo.
// Problem under test: Fan out video.published to audit+UI
```

Trade-offs

- Use Observer when «Fan out video.published to audit+UI» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Observer in product language.

Related Patterns

- Name the pattern usually confused with Observer and separate the forces.
- Sketch the roles for «Fan out video.published to audit+UI» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Fan out video.published to audit+UI
- Naive pain → Observer boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.